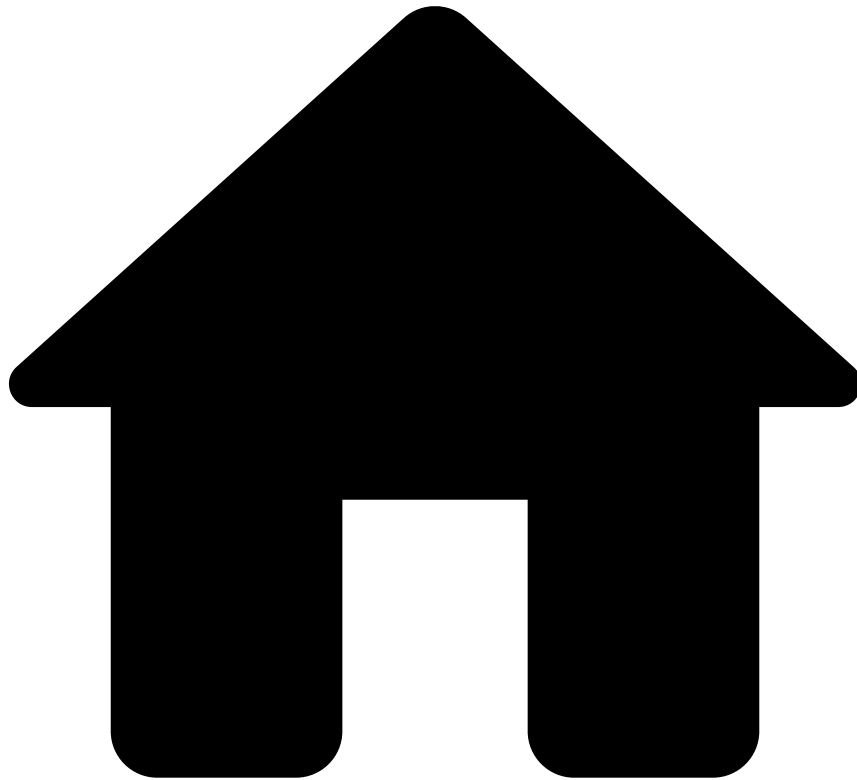


•



- [Core Concepts](#)
- [Models](#)
- [Model Training](#)
- FAQ on Gradient Boosting Frameworks

On this page

FAQ on Gradient Boosting Frameworks

Was this page helpful? 

This section contains answers to common questions when configuring a gradient boosting framework in Pulse.

What is string indexing?

String indexing goes by many other names as well, such as label encoding and integer encoding. For an overview of this technique, you can read [this article](#).

What is considered too many categories?🔗

According to Microsoft LightGBM's recommendations, you should not have too many categories in a categorical feature. This could mean hundreds or thousands, as it depends on the model. For performance, categorical values should not be limited to <1% of the training instances.

If there are too many categories, the recommendation is to index them and declare that feature as numerical. However, LightGBM / FairGBM will then assume such values **are** sorted. This means the mapping you choose matters. We recommend using frequency-based or risk-based indexing.

This choice implies that you must do the string indexing yourself, by adding a step to the features plan. In this step, take a mapping (through a list operator or user-defined function that loads the mapping from disk) and apply it to the input feature to generate the numerical feature to be used by the LightGBM / FairGBM model.

Remember that this indexing must work reliably and in the same manner in any environment, be it data science phase, or QA, or Production, to avoid compromising the model features and behavior.

There are cases with performance improvements where up to 10,000 categories were indexed through frequency for a categorical feature. Declaring that feature as numerical decreased performance around 1% recall.

As with all data science tasks, you should experiment and figure out how many categories give you the best results for your project.

Does Pulse's categorical features indexing algorithm impact my model?🔗

This particular way of categorical ordering by Pulse (lexicographic order) does not affect your model results because LightGBM / FairGBM assumes the categorical features have no order, and sorts the different categories prior to splitting by the objective function.

The essential part is that the indexing is the same during training and scoring (which Pulse does).

How are missing/null values handled?🔗

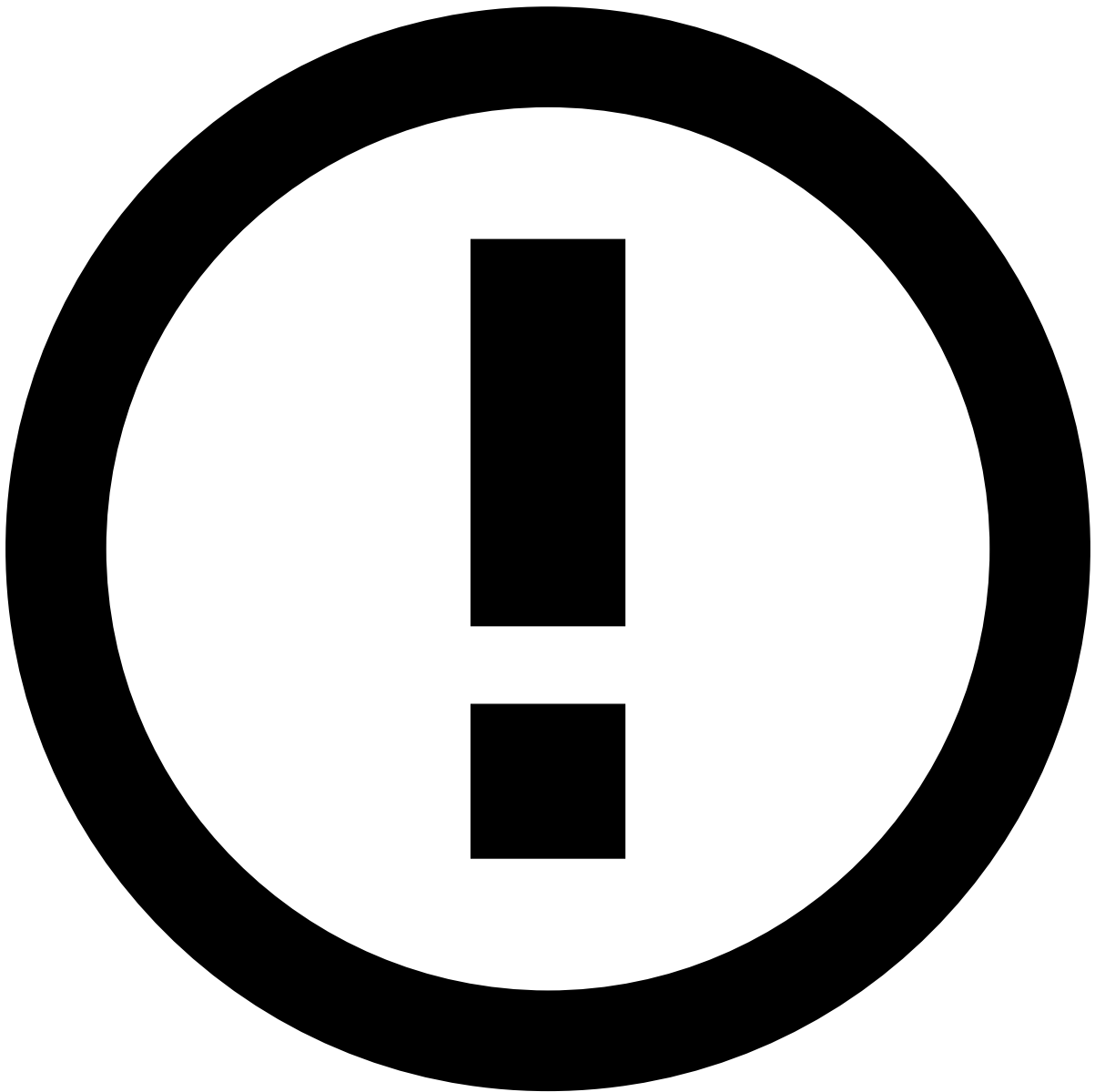
LightGBM and FairGBM handle nulls properly. In Pulse this is done by mapping any null / missing value to a NaN value in the backend, which is then interpreted as missing (you cannot disable this level of missing value handling in Pulse by design).

It is possible to consider 0 values as missing by enabling "Use zeroes as missing".

LightGBM and FairGBM handle splits and missing values in scoring using the following rules:

- For categorical features:
 - Categorical splits are performed in "one" vs "others" approach, with the split category of interest going to the left of the split, and the remaining categories go to the "others" side, on the right side of the split.
 - A missing value in a categorical feature split allots the instance to the right side of the split. This is the "other" categories side.
- For numerical features:
 - A numerical split places numbers smaller than the split point to the left, and larger ones to the right of it.
 - *If there were no missing values during model training*, the missing value in scoring is replaced by 0 and the splits are followed accordingly.
 - *If there were missing values during model training for that split*: missing values are allocated to the side of the split that reduces training loss the most (during prediction that side is chosen as well).

See [this thread](#) in the LightGBM forum for more details.



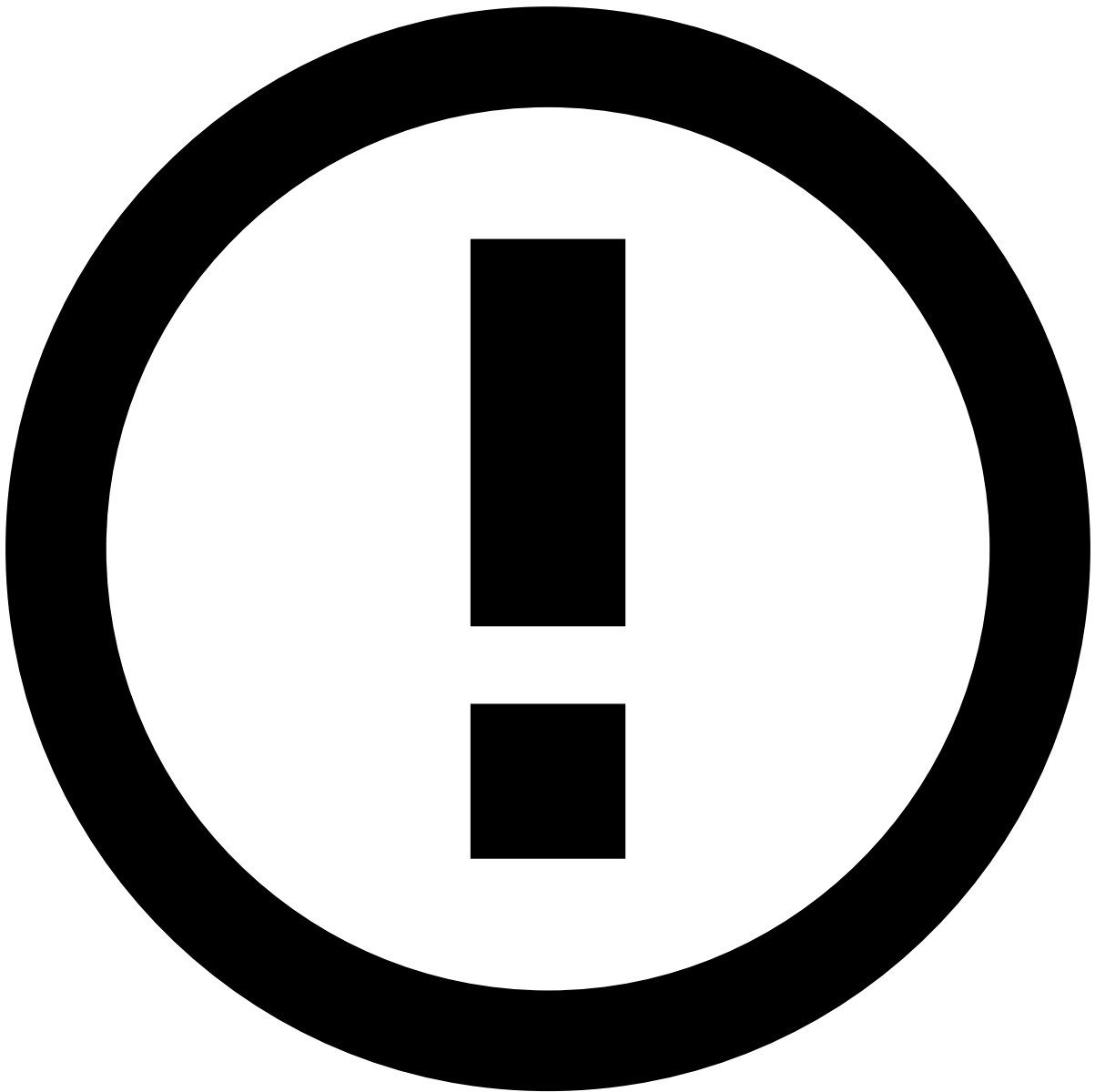
info

FairGBM does not handle missing sensitive attribute values.

What are the training speeds for the different boosting types?[🔗](#)

There aren't any benchmarks available online. However, by [modifying the benchmarks](#) and testing for the HIGGS dataset, it is possible to get an idea of the speed difference on a large dataset.

Feedzai has conducted tests for LightGBM. See the test results below.



info

Note that, for FairGBM, extra computations make it slower than LightGBM during train. For this reason, such results do not apply to FairGBM.

Training time for the Higgs dataset

The Higgs dataset is a dataset with 28 features and 11M rows, of which 10.5M rows are used for the model training. The tests were conducted on a Dell laptop with 4 cores (x2 because of hyper-threading) and 32G RAM.

`boosting_type=GBDT`

Measurement	Measurement value
Real	10m 12s
User	51m 57s
System	1m 8s

boosting_type=GOSS

Measurement	Measurement value
Real	8m 17s
User	42m 59s
System	1m 1s

boosting_type=DART

Measurement	Measurement value
Real	60m 9s
User	363m 31s
System	2m 8s

Conclusions

As it is a single dataset and machine under comparison, these results serve only to give an idea of the different boosting model training speeds.

This comparison in particular shows roughly a **20% reduction** in training time whilst using **GOSS** boosting type instead of **GBDT**.

The biggest difference comes from using `boosting_type=DART` , which is roughly **6x slower** than **GBDT** in this case.